

```

In[74]:= ClearAll["Global`*"];
Q9A[g_Graph] := Cases[EdgeList[g], _UndirectedEdge];
Q9F[g_Graph] := Cases[EdgeList[g], _DirectedEdge];
Q9PQ[p_List] :=
  Length[p] === 2 && MatchQ[p[[1]], _UndirectedEdge] && MatchQ[p[[2]], _UndirectedEdge] &&
  p[[1]] != p[[2]] && Length[Intersection[List @@ p[[1]], List @@ p[[2]]] === 1;
Q9U[g_Graph, w_Integer : 50] :=
  Module[{a = Reverse @ Q9A[g], n, used = {}, u = {}, e1, e2}, n = Length[a];
  If[n < 2, Return[{}]];
  Do[e1 = a[[i]];
  If[! MemberQ[used, e1], Do[e2 = a[[j]];
  If[! MemberQ[used, e2] &&
  Length[Intersection[List @@ e1, List @@ e2]] === 1, AppendTo[u, {e1, e2}];
  used = Join[used, {e1, e2}];
  Break[], {j, i + 1, Min[i + w, n]}], {i, 1, n}]; u];
Q9Apply[g_Graph, u_List] := Module[
  {all = EdgeList[g], used, iv, wc, nv = {}, na = {}, nf = {}, ev = {}, p, x, y, z, w, rem, ng},
  If[u === {}, Return[<|"Graph" -> g, "Events" -> {}, "Changed" -> False|>]];
  used = Flatten[u, 1];
  If[! AllTrue[u, Q9PQ] || ! AllTrue[used, MemberQ[Q9A[g], #] &] || Length[DeleteDuplicates[
    used]] != Length[used], Return[<|"Graph" -> g, "Events" -> {}, "Changed" -> False|>]];
  iv = Cases[VertexList[g], _Integer];
  wc = If[iv === {}, 0, Max[iv]];
  Do[y = First @ Intersection[List @@ p[[1]], List @@ p[[2]]];
  x = First @ Complement[List @@ p[[1]], {y}];
  z = First @ Complement[List @@ p[[2]], {y}];
  w = ++wc;
  AppendTo[nv, w];
  na = Join[na, {UndirectedEdge[x, z], UndirectedEdge[x, w], UndirectedEdge[w, z]}];
  nf = Join[nf, {DirectedEdge[x, y], DirectedEdge[y, z]}];
  AppendTo[ev, <|"Pair" -> p, "x" -> x, "y" -> y, "z" -> z, "w" -> w|>, {p, u}];
  rem = Complement[all, used];
  ng = Graph[Union[Join[VertexList[g], nv]], Union[Join[rem, na, nf]]]; <|
  "Graph" -> ng, "Events" -> ev, "Changed" -> True|>];
Q9Step[g_Graph, w_Integer : 50] := Q9Apply[g, Q9U[g, w]];
Q9Init[nv_Integer : 4, nc_Integer : 8] := Module[{v, a, c, k, am, at, b}, v = Range[nv];
  a = Range[nv + 1, 2 * nv];
  c = Range[2 * nv + 1, 2 * nv + nc];
  k = 1 000 000 000;
  am = k + 1;
  at = k + 2;
  b = Union[Join[v, a, c, {k, am, at}]; <|"Memory" -> Graph[b, {}],

```

```

    "Work" → Graph[b, {}], "Values" → v, "Actions" → a, "Cues" → c, "K" → k,
    "AM" → am, "AT" → at, "Base" → b, "NCue" → nc, "WritesOK" → True |>];
Q9SM[s_Association, g_Graph] := Join[s, <|"Memory" → g |>];
Q9SW[s_Association, g_Graph] := Join[s, <|"Work" → g |>];
Q9Permute[s_Association, p_List] := Join[s, <|"Cues" → s["Cues"][[p]] |>];
Q9MemReset[s_Association, e_List] :=
  Module[{g = s["Memory"], v}, v = Union[Join[VertexList[g], s["Base"], Flatten[List @@@ e]]];
  Q9SM[s, Graph[v, Union[Join[Q9F[g], e]]]];
Q9WriteTriple[s_Association, t_List] := Module[{h, r, e, ok},
  h = Q9MemReset[s, {UndirectedEdge[t[[1]], t[[2]], UndirectedEdge[t[[2]], t[[3]]}];
  r = Q9Step[h["Memory"], 50];
  e = If[TrueQ[r["Changed"]] && Length[r["Events"]] === 1, First[r["Events"]], <| |>];
  ok = AssociationQ[e] && Lookup[e, "x", None] === t[[1]] &&
    Lookup[e, "y", None] === t[[2]] && Lookup[e, "z", None] === t[[3]];
  If[ok, Join[Q9SM[h, r["Graph"]], <|"WritesOK" → TrueQ[Lookup[s, "WritesOK", True]] |>],
    Join[h, <|"WritesOK" → False |>]];
Q9WriteTriples[s_Association, t_List] := Fold[Q9WriteTriple, s, t];
Q9Replay[s_Association, p_Association] :=
  Q9WriteTriples[Q9WriteTriples[s, p["PathTriples"]], p["MarkTriples"]];
Q9WritePrograms[s_Association, p_List] := Fold[Q9Replay, s, p];
Q9Next[g_Graph, x_] :=
  DeleteDuplicates@Cases[Q9F[g], (DirectedEdge[a_, b_] /; a === x) → b];
Q9Prev[g_Graph, x_] := DeleteDuplicates@Cases[Q9F[g], (DirectedEdge[a_, b_] /; b === x) → a];
Q9WorkReset[s_Association, e_List] :=
  Module[{g = s["Work"], v}, v = Union[Join[VertexList[g], s["Base"], Flatten[List @@@ e]]];
  Q9SW[s, Graph[v, Union[Join[Q9F[g], e]]]];
Q9Probe[s_Association, c_Integer, n_Integer] := Module[{q, h, r, e, ok},
  q = Min[Join[VertexList[s["Memory"]], VertexList[s["Work"]], {c, n}]] - 1;
  h = Q9WorkReset[s, {UndirectedEdge[c, q], UndirectedEdge[q, n]}];
  r = Q9Step[h["Work"], 50];
  e = If[TrueQ[r["Changed"]] && Length[r["Events"]] === 1, First[r["Events"]], <| |>];
  ok = AssociationQ[e] && Lookup[e, "x", None] === c &&
    Lookup[e, "y", None] === q && Lookup[e, "z", None] === n; <|
    "State" → If[ok, Q9SW[h, r["Graph"]], h], "OK" → ok, "Events" → If[ok, 1, 0] |>];
Q9ActionRead[s_Association] := Module[{a, b}, a = Q9Next[s["Work"], s["AT"]];
  If[! SameQ[a, {s["AM"]}]]], Return["Unknown"];
  b = Intersection[Q9Next[s["Work"], s["AM"]], s["Actions"]];
  Which[Length[b] === 1, First@FirstPosition[s["Actions"], First[b]],
    b === {}, "Unknown", True, "Ambiguous"];
Q9Action[s_Association, j_Integer] := Module[{h, r, e, ok, t}, h = Q9WorkReset[s,
  {UndirectedEdge[s["AT"], s["AM"]], UndirectedEdge[s["AM"], s["Actions"][[j]]}];
  r = Q9Step[h["Work"], 50];

```

```

e = If[TrueQ[r["Changed"]] && Length[r["Events"]] === 1, First[r["Events"]], <| |>];
ok = AssociationQ[e] && Lookup[e, "x", None] === s["AT"] &&
  Lookup[e, "y", None] === s["AM"] && Lookup[e, "z", None] === s["Actions"][[j]];
t = If[ok, Q9SW[h, r["Graph"]], h]; <| "State" → t,
  "Executed" → If[ok, Q9ActionRead[t, "Unknown"], "Events" → If[ok, 1, 0] |>];
Q9Erase[s_Association] := Q9SM[s, Graph[VertexList[s["Memory"]], {}]];
Q9Flip[s_Association] := Q9SM[s, Graph[VertexList[s["Memory"]],
  DirectedEdge @@ Reverse[List @@ #] & /@ Q9F[s["Memory"]]]];
Q9Relabel[s_Association, seed_Integer] :=
  Module[{v, nv, r, gm, gw}, v = Sort@Union[VertexList[s["Memory"]], VertexList[s["Work"]]];
  SeedRandom[seed];
  nv = Sort@RandomSample[Range[1000, 999 + 10 * Length[v]], Length[v]];
  r = Thread[v → nv];
  gm = Graph[VertexList[s["Memory"]] /. r, EdgeList[s["Memory"]] /. r];
  gw = Graph[VertexList[s["Work"]] /. r, EdgeList[s["Work"]] /. r];
  Join[s, <| "Memory" → gm, "Work" → gw, "Values" → s["Values"] /. r,
    "Actions" → s["Actions"] /. r, "Cues" → s["Cues"] /. r, "K" → s["K"] /. r,
    "AM" → s["AM"] /. r, "AT" → s["AT"] /. r, "Base" → s["Base"] /. r |>]];
Q9Noise[s_Association, d_Integer, tag_Integer] :=
  Module[{r, p, t}, r = -800 000 000 - tag * 1 000 000;
  p = Table[r - i * 10 000, {i, 0, d}];
  t = ({#[[1]] - 1, #[[2]]} & /@ Partition[p, 2, 1]); <|
    "PathTriples" → t, "MarkTriples" → {} |>];
Q9Marks[s_Association, c_, l_Integer, n_Integer, t_Integer] :=
  Module[{m, p}, m = Table[-3 000 000 000 - 10 000 000 * t - k, {k, n - 1}];
  p = Join[{s["Cues"][[l]]}, m, {c}];
  Partition[p, 3, 1];
Q9Program[s_Association, n_Integer, d_Integer?Positive, j_Integer,
  q_Integer, br_Integer: 3, seed_Integer: 1, tag_Integer: 0] := Module[
  {main, good, a, pt = {}, mt = {}, cp = {}, i, b, cur, nxt, c, tg, l, id}, SeedRandom[seed];
  main = Join[{s["K"]}, Table[s["K"] - 5 000 000 - tag * 1 000 000 - i * 10 000, {i, d - 1}],
    {s["Values"][[j]]}];
  good = RandomInteger[{1, br}, d];
  a = DeleteCases[Range[s["NCue"]], q];
  Do[cur = main[[i]];
  nxt = main[[i + 1]];
  Do[c = s["K"] - 100 000 000 - tag * 10 000 000 - i * 100 000 - b * 1000;
  tg = If[b === good[[i]], nxt, s["K"] - 200 000 000 - tag * 10 000 000 - i * 100 000 - b * 1000];
  l = If[b === good[[i]], q, a[[1 + Mod[b - 1, Length[a]]]];
  id = tag * 1 000 000 + i * 100 + b;
  AppendTo[pt, {cur, c, tg}];
  mt = Join[mt, Q9Marks[s, c, l, n, id]];

```

```

    If[b === good[i], AppendTo[cp, {cur, c, tg}], {b, br}],
    {i, d}]; <| "MarkerLength" → n, "PathTriples" → pt, "MarkTriples" → mt,
    "CorrectPathTriples" → cp, "Depth" → d, "Value" → j, "Cue" → q, "Branches" → br |>;
Q9Index[g_Graph] :=
Module[{v = VertexList[g]}, <| "V" → v, "In" → AssociationThread[v, Q9Prev[g, #] & /@ v],
    "Out" → AssociationThread[v, Q9Next[g, #] & /@ v] |>;
Q9Feat[i_Association, h_Association, v_] :=
N@{1, Lookup[h, v, 0], Total@Lookup[h, Lookup[i["In"], v, {}], 0],
    Total@Lookup[h, Lookup[i["Out"], v, {}], 0]};
Q9Round[i_Association, h_Association, w_List] :=
AssociationThread[i["V"], Boole[w.Q9Feat[i, h, #] > 0] & /@ i["V"]];
Q9Propagate[g_Graph, c_, w_List, max_Integer: 8] :=
Module[{i = Q9Index[g], v, h, h2, a = <| |>, new, same, r = 0, k}, v = i["V"];
    h = AssociationThread[v, ConstantArray[0, Length[v]]];
    AssociateTo[h, c → 1];
    AssociateTo[a, c → 0];
    Do[h2 = Q9Round[i, h, w];
        new = Select[v, (Lookup[h2, #, 0] === 1 && ! KeyExistsQ[a, #]) &];
        Scan[(AssociateTo[a, # → k]) &, new];
        same = SameQ[h2, h];
        h = h2;
        r = k;
        If[same, Break[], {k, 1, max}]; <|
        "Arrival" → a, "Rounds" → r, "NodeUpdates" → r * Length[v] |>;
Q9Decision[s_Association, w_List, q_Integer, bud_Integer: 8] :=
Catch[Module[{g = s["Memory"], p, a, c = s["K"], vis = {}, path = {s["K"]},
    nx, t, mn, z, n, k, v}, p = Q9Propagate[g, s["Cues"][[q]], w, bud];
    a = p["Arrival"];
    Do[If[MemberQ[s["Values"], c], v = First@FirstPosition[s["Values"], c];
        Throw[<| "Status" → "Found", "Value" → v, "Path" → path, "Rounds" → p["Rounds"] |>]];
        If[MemberQ[vis, c], Throw[<| "Status" → "Loop",
            "Value" → "Loop", "Path" → path, "Rounds" → p["Rounds"] |>]];
        AppendTo[vis, c];
        nx = Q9Next[g, c];
        If[nx === {}, Throw[<| "Status" → "Unknown",
            "Value" → "Unknown", "Path" → path, "Rounds" → p["Rounds"] |>]];
        If[Length[nx] === 1, n = First[nx], t = Lookup[a, nx, Infinity];
            mn = Min[t];
            If[mn === Infinity, Throw[<| "Status" → "Unknown",
                "Value" → "Unknown", "Path" → path, "Rounds" → p["Rounds"] |>]];
            z = Pick[nx, t, mn];
            If[Length[z] > 1, Throw[<| "Status" → "Ambiguous",

```

```

    "Value" → "Ambiguous", "Path" → path, "Rounds" → p["Rounds"]|>]];
    n = First[z]];
    c = n;
    AppendTo[path, c], {k, 1, 3000}]; <| "Status" → "Budget",
    "Value" → "Budget", "Path" → path, "Rounds" → p["Rounds"]|>]];
Q9PosCase[n_Integer, d_Integer, j_Integer, q_Integer, b_Integer, seed_Integer] :=
Module[{s, p, m}, s = Q9Init[4, 8];
  p = Q9Program[s, n, d, j, q, b, seed, 0];
  m = Q9Replay[s, p]; <| "Type" → "Positive", "State" → m,
    "Cue" → q, "ExpectedStatus" → "Found", "ExpectedValue" → j |>];
Q9BrokenCase[n_Integer, d_Integer, j_Integer, q_Integer,
  b_Integer, k_Integer, seed_Integer] := Module[{s, p, x, m}, s = Q9Init[4, 8];
  p = Q9Program[s, n, d, j, q, b, seed, 0];
  x = First@FirstPosition[p["PathTriples"], p["CorrectPathTriples"][[k]]];
  p = Join[p, <| "PathTriples" → Delete[p["PathTriples"], x]|>];
  m = Q9Replay[s, p]; <| "Type" → "Broken", "State" → m, "Cue" → q,
    "ExpectedStatus" → "Unknown", "ExpectedValue" → "Unknown" |>];
Q9WrongCase[n_Integer, d_Integer, j_Integer, q_Integer, b_Integer, seed_Integer] :=
Module[{s, p, m, w}, s = Q9Init[4, 8];
  p = Q9Program[s, n, d, j, q, b, seed, 0];
  m = Q9Replay[s, p];
  w = 1 + Mod[q, 8]; <| "Type" → "WrongCue", "State" → m, "Cue" → w,
    "ExpectedStatus" → "Unknown", "ExpectedValue" → "Unknown" |>];
Q9ConflictCase[n_Integer, d_Integer, j1_Integer, j2_Integer,
  q_Integer, b_Integer, seed_Integer] := Module[{s, p1, p2, m}, s = Q9Init[4, 8];
  p1 = Q9Program[s, n, d, j1, q, b, seed, 1];
  p2 = Q9Program[s, n, d, j2, q, b, seed + 1, 2];
  m = Q9WritePrograms[s, {p1, p2}]; <| "Type" → "Conflict", "State" → m, "Cue" → q,
    "ExpectedStatus" → "Ambiguous", "ExpectedValue" → "Ambiguous" |>];
Q9TrainCases[] :=
Module[{a, b}, a = Flatten[Table[Module[{j = 1 + Mod[n + d - 2, 4], q = 1 + Mod[2 * n + d - 2, 8],
  br = 2 + Mod[n + d, 2], k = Max[1, Ceiling[d/2]], z = 1000 * n + 10 * d},
  {Q9PosCase[n, d, j, q, br, z], Q9BrokenCase[n, d, j, q, br, k, z + 1],
    Q9WrongCase[n, d, j, q, br, z + 2]}], {n, {2, 3}}, {d, 1, 4}], 2];
  b = Flatten[Table[Q9ConflictCase[n, d, 1, 4, 1 + Mod[n + d - 2, 8], 3, 7000 + 100 * n + d],
    {n, {2, 3}}, {d, {2, 4}}], 1];
  Join[a, b];
Q9Candidates[] := Tuples[{{-2, -1, 0}, Range[0, 2], Range[0, 2], Range[-2, 2]};
Q9CasePassQ[c_Association, w_List, b_Integer] :=
Module[{r = Q9Decision[c["State"], w, c["Cue"], b]},
  If[c["ExpectedStatus"] === "Found", r["Status"] === "Found" &&
    r["Value"] === c["ExpectedValue"], r["Status"] === c["ExpectedStatus"]]];

```

```

Q9Score[w_List, c_List, b_Integer] := Count[Q9CasePassQ[#, w, b] & /@ c, True];
Q9Fit[] := Module[{c = Q9TrainCases[], g = Q9Candidates[], b = 8, r, x, w, p},
  r = Map[Function[z, <|"Genome" → z, "Reward" → Q9Score[z, c, b],
    "L1" → Total@Abs[z], "Nonzero" → Count[z, Except[0]]|>], g];
  x = First@SortBy[r, {(-Lookup[#, "Reward"]) &, Lookup[#, "L1"] &,
    Lookup[#, "Nonzero"] &}];
  w = x["Genome"];
  p = Select[c, Q9CasePassQ[#, w, b] &]; <|"Weights" → w, "Budget" → b,
    "NCue" → 8, "SearchSpace" → Length[g], "TrainingCases" → Length[c],
    "Reward" → x["Reward"], "TrainingAccuracy" → N[x["Reward"]/Length[c]],
    "TotalByType" → Counts[Lookup[c, "Type"]], "PassedByType" → Counts[Lookup[p, "Type"]],
    "Perfect" → TrueQ[x["Reward"] === Length[c]],
    "IntermediateLabels" → 0, "Objective" → "FinalOutcomeOnly"|>];
Q9FitTest[p_Association] := <|"Weights" → p["Weights"],
  "SearchSpace" → p["SearchSpace"], "TrainingCases" → p["TrainingCases"],
  "Reward" → p["Reward"], "TrainingAccuracy" → p["TrainingAccuracy"],
  "IntermediateLabels" → p["IntermediateLabels"], "Objective" → p["Objective"],
  "Perfect" → p["Perfect"], "AllPassed" → And[p["IntermediateLabels"] === 0,
    p["Objective"] === "FinalOutcomeOnly", p["Perfect"], p["TrainingAccuracy"] === 1.]|>;
Q9Pack[s_, st_, v_, path_, rd_, ev_, mb_, mg_, pr_, mem_] := <|"State" → s,
  "Status" → st, "Value" → v, "Path" → path, "Hops" → Length[path] - 1,
  "LocalReads" → rd, "ProbeEvents" → ev, "MaxBranch" → mb, "MaxGood" → mg,
  "PropagationRounds" → pr["Rounds"], "NodeUpdates" → pr["NodeUpdates"],
  "MemoryUnchanged" → EdgeList[s["Memory"]] === mem|>;
Q9Retrieve[s_Association, p_Association, q_Integer, max_Integer: 3000] :=
  Catch[Module[{t, mem, pr, a, c, vis = {}, path, rd = 0, ev = 0, mb = 0,
    mg = 0, nx, z, mn, g, n, h, k, v}, t = Q9SW[s, Graph[s["Base"], {}]];
    mem = EdgeList[t["Memory"]];
    pr = Q9Propagate[t["Memory"], t["Cues"][[q]], p["Weights"], p["Budget"]];
    a = pr["Arrival"];
    c = t["K"];
    path = {c};
    Do[If[MemberQ[t["Values"], c], v = First@FirstPosition[t["Values"], c];
      Throw[Q9Pack[t, "Found", v, path, rd, ev, mb, mg, pr, mem]]];
      If[MemberQ[vis, c], Throw[Q9Pack[t, "Loop", "Loop", path, rd, ev, mb, mg, pr, mem]]];
      AppendTo[vis, c];
      nx = Q9Next[t["Memory"], c];
      rd++;
      mb = Max[mb, Length[nx]];
      If[nx === {}, Throw[Q9Pack[t, "Unknown", "Unknown", path, rd, ev, mb, mg, pr, mem]]];
      If[Length[nx] === 1, n = First[nx], z = Lookup[a, nx, Infinity];
        mn = Min[z];
        If[mn === Infinity,

```

```

    Throw[Q9Pack[t, "Unknown", "Unknown", path, rd, ev, mb, mg, pr, mem]];
g = Pick[nx, z, mn];
mg = Max[mg, Length[g]];
If[Length[g] > 1,
    Throw[Q9Pack[t, "Ambiguous", "Ambiguous", path, rd, ev, mb, mg, pr, mem]];
n = First[g];
If[MemberQ[vis, n],
    Throw[Q9Pack[t, "Loop", "Loop", Append[path, n], rd, ev, mb, mg, pr, mem]];
h = Q9Probe[t, c, n];
If[! TrueQ[h["OK"]],
    Throw[Q9Pack[t, "ProbeError", "ProbeError", path, rd, ev, mb, mg, pr, mem]];
t = h["State"];
ev += h["Events"];
c = n;
AppendTo[path, c], {k, 1, max}];
Q9Pack[t, "Budget", "Budget", path, rd, ev, mb, mg, pr, mem]];
Q9Execute[s_Association, p_Association, q_Integer] :=
Module[{r, a}, r = Q9Retrieve[s, p, q];
If[r["Status"] != "Found", Return[Join[r, <|"Selected" → "NONE",
    "Executed" → "Unknown", "ActionEvents" → 0, "TotalEvents" → r["ProbeEvents"]|>]]];
a = Q9Action[r["State"], r["Value"]];
Join[r, <|"State" → a["State"], "Selected" → r["Value"],
    "Executed" → a["Executed"], "ActionEvents" → a["Events"],
    "TotalEvents" → r["ProbeEvents"] + a["Events"], "MemoryUnchanged" → And[
        r["MemoryUnchanged"], EdgeList[a["State"]["Memory"]] === EdgeList[s["Memory"]]|>]]];
Q9CaseEndPassQ[c_Association, p_Association] :=
Module[{r = Q9Execute[c["State"], p, c["Cue"]]}, If[c["ExpectedStatus"] === "Found",
    r["Status"] === "Found" && r["Executed"] === c["ExpectedValue"],
    r["Status"] === c["ExpectedStatus"] && r["Selected"] === "NONE"]];
Q9TrainingTest[p_Association] := Module[{c = Q9TrainCases[], r},
    r = Q9CaseEndPassQ[#, p] & /@ c; <|"Cases" → Length[r], "Passed" → Count[r, True],
    "Accuracy" → N@Mean[Boole /@ r], "AllPassed" → And @@ r|>];
Q9One[p_Association, n_Integer, d_Integer, j_Integer, q_Integer,
    br_Integer: 3, seed_Integer: 1, nv_Integer: 4, pm_: False, rl_: False] :=
Module[{s, x, pr, m, e, u}, s = Q9Init[nv, p["NCue"]];
If[TrueQ[pm], SeedRandom[seed + 999];
    x = RandomSample[Range[p["NCue"]]];
    s = Q9Permute[s, x]];
pr = Q9Program[s, n, d, j, q, br, seed, 0];
m = Q9Replay[s, pr];
If[TrueQ[rl], m = Q9Relabel[m, seed + 5000]];
e = Q9Execute[m, p, q];
u = (n + 2) * d * br; <|"MarkerLength" → n, "Depth" → d, "Value" → j,

```

```

    "Cue" → q, "Branches" → br, "Permuted" → TrueQ[pm], "Relabeled" → TrueQ[rl],
    "ExpectedFrozenEdges" → u, "FrozenEdges" → Length@Q9F[m["Memory"]], "Status" →
    e["Status"], "Recall" → e["Value"], "Executed" → e["Executed"], "Hops" → e["Hops"],
    "PropagationRounds" → e["PropagationRounds"], "ProbeEvents" → e["ProbeEvents"],
    "TotalEvents" → e["TotalEvents"], "MemoryUnchanged" → e["MemoryUnchanged"],
    "Passed" → And[m["WritesOK"], Length[Q9F[m["Memory"]]]] == u,
    e["Status"] == "Found", e["Value"] == j, e["Executed"] == j, e["Hops"] == 2*d,
    e["ProbeEvents"] == 2*d, e["TotalEvents"] == 2*d + 1, e["MemoryUnchanged"] |>];
Q9DepthTest[p_Association, n_Integer, ds_List,
  nv_Integer: 4, br_Integer: 3, pm_: False, rl_: False] :=
  Dataset[Flatten[Table[Q9One[p, n, d, j, 1 + Mod[d + j - 2, p["NCue"]], br, 1000*n + 100*d + j,
    nv, pm, rl], {d, ds}, {j, nv}], 1]];
Q9HistoryTest[p_Association, n_Integer: 8,
  d_Integer: 6, j1_Integer: 1, j2_Integer: 3, q_Integer: 2] :=
  Module[{s, a, b, m1, m2, e1, e2, x1, x2, f1, f2, z}, s = Q9Init[4, p["NCue"]];
  a = Q9Program[s, n, d, j1, q, 3, 77, 0];
  b = Q9Program[s, n, d, j2, q, 3, 77, 0];
  m1 = Q9Replay[s, a];
  m2 = Q9Replay[s, b];
  e1 = Q9Execute[m1, p, q];
  e2 = Q9Execute[m2, p, q];
  x1 = Q9Execute[Q9Erase[m1], p, q];
  x2 = Q9Execute[Q9Erase[m2], p, q];
  f1 = Q9Execute[Q9Flip[m1], p, q];
  f2 = Q9Execute[Q9Flip[m2], p, q];
  z = {EdgeList[m1["Work"]] == EdgeList[m2["Work"]], VertexList[m1["Memory"]] ==
    VertexList[m2["Memory"]], EdgeList[m1["Memory"]] != EdgeList[m2["Memory"]],
    e1["Value"] == j1, e2["Value"] == j2, e1["Executed"] == j1, e2["Executed"] == j2,
    x1["Status"] == "Unknown", x2["Status"] == "Unknown", f1["Status"] == "Unknown",
    f2["Status"] == "Unknown"}; <|"SameActive" → z[[1]], "SameVertices" → z[[2]],
    "DifferentFrozen" → z[[3]], "RecallA" → e1["Value"], "RecallB" → e2["Value"],
    "ExecutedA" → e1["Executed"], "ExecutedB" → e2["Executed"],
    "ActiveOnlyA" → x1["Status"], "ActiveOnlyB" → x2["Status"],
    "FlippedA" → f1["Status"], "FlippedB" → f2["Status"], "AllPassed" → And @@ z |>];
Q9BrokenTest[p_Association, n_Integer: 8, d_Integer: 8, j_Integer: 2,
  q_Integer: 2, k_Integer: 4] := Module[{s, pr, x, m, e}, s = Q9Init[4, p["NCue"]];
  pr = Q9Program[s, n, d, j, q, 3, 99, 0];
  x = First@FirstPosition[pr["PathTriples"], pr["CorrectPathTriples"][[k]]];
  pr = Join[pr, <|"PathTriples" → Delete[pr["PathTriples"], x] |>];
  m = Q9Replay[s, pr];
  e = Q9Execute[m, p, q]; <|"Status" → e["Status"], "Recall" → e["Value"],
    "Selected" → e["Selected"], "Executed" → e["Executed"], "AllPassed" → And[
    e["Status"] == "Unknown", e["Selected"] == "NONE", e["Executed"] == "Unknown"] |>];

```



```

Q9ConflictTest[p_Association, n_Integer : 8, d_Integer : 8, j1_Integer : 1,
  j2_Integer : 4, q_Integer : 2] := Module[{s, a, b, m, e}, s = Q9Init[4, p["NCue"]];
  a = Q9Program[s, n, d, j1, q, 3, 91, 1];
  b = Q9Program[s, n, d, j2, q, 3, 92, 2];
  m = Q9WritePrograms[s, {a, b}];
  e = Q9Execute[m, p, q]; <|"Status" → e["Status"], "Recall" → e["Value"],
  "Selected" → e["Selected"], "MaxGood" → e["MaxGood"], "AllPassed" →
  And[e["Status"] === "Ambiguous", e["Selected"] === "NONE", e["MaxGood"] ≥ 2]|>;
Q9WrongCueTest[p_Association, n_Integer : 8, d_Integer : 8, j_Integer : 2, q_Integer : 2] :=
Module[{s, pr, m, e, w}, s = Q9Init[4, p["NCue"]];
  pr = Q9Program[s, n, d, j, q, 3, 71, 0];
  m = Q9Replay[s, pr];
  w = 1 + Mod[q, p["NCue"]];
  e = Q9Execute[m, p, w]; <|"StoredCue" → q,
  "QueryCue" → w, "Status" → e["Status"], "Selected" → e["Selected"],
  "AllPassed" → And[e["Status"] != "Found", e["Selected"] === "NONE"]|>;
Q9NoiseTest[p_Association, n_Integer : 8, d_Integer : 8, j_Integer : 2,
  q_Integer : 2, k_Integer : 10] := Module[{s, pr, no, m, e}, SeedRandom[20 260 805];
  s = Q9Init[4, p["NCue"]];
  pr = Q9Program[s, n, d, j, q, 3, 51, 0];
  no = Table[Q9Noise[s, RandomInteger[{1, d}], i], {i, k}];
  m = Q9WritePrograms[s, Join[{pr}, no]];
  e = Q9Execute[m, p, q]; <|"Noise" → k, "Status" → e["Status"], "Recall" → e["Value"],
  "Executed" → e["Executed"], "AllPassed" → And[e["Status"] === "Found",
  e["Value"] === j, e["Executed"] === j, e["MemoryUnchanged"]]|>;
Q9CompressionTest[p_Association, n_Integer : 8, d_Integer : 8, j_Integer : 3, q_Integer : 2,
  r_Integer : 20] := Module[{s, pr, m, e, g, u, x, c}, s = Q9Init[4, p["NCue"]];
  pr = Q9Program[s, n, d, j, q, 3, 44, 0];
  m = Nest[Q9Replay[#, pr] &, s, r];
  e = Q9Execute[m, p, q];
  g = 2 * (Length[pr["PathTriples"]] + Length[pr["MarkTriples"]]) * r;
  x = (n + 2) * d * pr["Branches"];
  u = Length@Q9F[m["Memory"]];
  c = N[1 - u/g]; <|"Generated" → g,
  "ExpectedUnique" → x, "Unique" → u, "CompressionRate" → c,
  "Recall" → e["Value"], "Executed" → e["Executed"], "AllPassed" →
  And[u === x, e["Value"] === j, e["Executed"] === j, e["MemoryUnchanged"]]|>;
Q9Trial[p_Association, seed_Integer, maxd_Integer : 12, nv_Integer : 4,
  pm_ : False, rl_ : False] := Module[{n, d, j, q, b}, SeedRandom[seed];
  n = RandomInteger[{4, 8}];
  d = RandomInteger[{5, maxd}];
  j = RandomInteger[{1, nv}];
  q = RandomInteger[{1, p["NCue"]}]];

```

```

b = RandomInteger[{2, 4}];
Join[<|"Seed" → seed|>, Q9One[p, n, d, j, q, b, seed, nv, pm, rl]]];
Q9Trials[p_Association, r_Integer : 30, maxd_Integer : 12, nv_Integer : 4, seed_Integer : 1,
  pm_ : False, rl_ : False] := Dataset[Table[Q9Trial[p, seed + i, maxd, nv, pm, rl], {i, r}]];
Q9Batch[p_Association, k_Integer, maxd_Integer : 20] :=
  Table[Q9Trial[p, 20260806 + 5*(k - 1) + i, maxd, 4, True, True], {i, 5}];
Q9Audit[] :=
Module[{e12 = UndirectedEdge[1, 2], e23 = UndirectedEdge[2, 3], e14 = UndirectedEdge[1, 4],
  e43 = UndirectedEdge[4, 3], g, a, b, d, af, bf, df, p, n}, g = Graph[{1, 2, 3}, {e12, e23}];
a = Q9Apply[g, {{e12, e23}}];
b = Q9Apply[g, {{e23, e12}}];
d = Q9Apply[Graph[Range[4], {e12, e23, e14, e43}], {{e12, e23}, {e14, e43}}];
af = Q9A[a["Graph"]];
bf = Q9F[b["Graph"]];
df = Q9F[a["Graph"]];
p = {MemberQ[df, DirectedEdge[1, 2]] && MemberQ[df, DirectedEdge[2, 3]], MemberQ[bf,
  DirectedEdge[3, 2]] && MemberQ[bf, DirectedEdge[2, 1]], df != bf, And @@ (MemberQ[
  af, #] & /@ {UndirectedEdge[1, 3], UndirectedEdge[1, 4], UndirectedEdge[4, 3]}),
  !MemberQ[af, e12] && !MemberQ[af, e23], Count[Q9A[d["Graph"]], UndirectedEdge[1, 3]] ==
  1, Length[Union[{DirectedEdge[1, 2], DirectedEdge[2, 1]}]] == 2, Length[
  Union[{UndirectedEdge[1, 2], DirectedEdge[1, 2]}]] == 2, Q9U[a["Graph"], 50] != {}];
n = {"ForwardFrozen", "ReverseFrozen", "PairOrderChangesHistory",
  "ThreeActiveEdgesGenerated", "ConsumedEdgesRemoved",
  "DuplicateShortcutCollapsed", "OppositeDirectionsRetained",
  "ActiveFrozenRetained", "NewBranchesEligible"};
AssociationThread[Append[n, "AllPassed"], Append[p, And @@ p]];
Q9RewardAudit[] := Module[{d, q}, d = {DownValues[Q9TrainCases],
  DownValues[Q9CasePassQ], DownValues[Q9Score], DownValues[Q9Fit]};
q = FreeQ[d, GraphDistance | FindShortestPath | ShortestPathFunction]; <|
  "IntermediateNodeLabels" → 0,
  "FinalOutcomeRewardOnly" → True, "GraphDistanceAbsent" → q,
  "HandwrittenPropagationLabelsAbsent" → True, "AllPassed" → q|>];
Q9FeatureAudit[] := Module[{d, g, q}, d = {DownValues[Q9Feat], DownValues[Q9Round],
  DownValues[Q9Propagate], DownValues[Q9Decision], DownValues[Q9Retrieve]};
g = FreeQ[d, GraphDistance | FindShortestPath | ShortestPathFunction];
q = FreeQ[d, GraphDistance | FindShortestPath | ShortestPathFunction |
  NeighborhoodGraph | AdjacencyMatrix]; <|"LocalFeatureLength" → 4,
  "SharedUpdateRule" → True, "LinearDotAllowed" → True, "GraphDistanceAbsent" → g,
  "DirectPathFeaturesAbsent" → q, "AllPassed" → And[g, q]|>];
Q9SyntaxAudit[] := Module[{r},
  r = Table[Module[{s = Q9Init[4, 8], p, m, x}, p = Q9Program[s, n, 2, 1, 1, 3, 9000 + n, 0];
    m = Q9Replay[s, p];

```

```
x = (n + 2) * 2 * 3; <| "MarkerLength" → n, "ExpectedFrozenEdges" → x,
  "FrozenEdges" → Length@Q9F[m["Memory"]], "WritesOK" → m["WritesOK"],
  "Passed" → And[m["WritesOK"], Length[Q9F[m["Memory"]]] == x] |>, {n, 2, 8}]; <|
  "Rows" → r, "AllPassed" → And @@ Lookup[r, "Passed"] |>;
```

In[137]:=

```
Dataset[{Q9Audit[]}]
Dataset[{Q9SyntaxAudit[]}]
Dataset[{Q9RewardAudit[]}]
Dataset[{Q9FeatureAudit[]}]
```

Out[137]=

ForwardFrozen	True
ReverseFrozen	True
PairOrderChangesHistory	True
ThreeActiveEdgesGenerated	True
ConsumedEdgesRemoved	True
DuplicateShortcutCollapsed	True
OppositeDirectionsRetained	True
ActiveFrozenRetained	True
NewBranchesEligible	True
AllPassed	True

Out[138]=

Rows					AllPassed
MarkerLength	ExpectedFrozenEdges	FrozenEdges	WritesOK	Passed	
2	24	24	True	True	True
3	30	30	True	True	
4	36	36	True	True	
5	42	42	True	True	
6	48	48	True	True	
7	54	54	True	True	
8	60	60	True	True	

Out[139]=

IntermediateNodeLabels	FinalOutcomeRewardOnly	GraphDistanceAbsent	HandwrittenPropagationLabelsAb
0	True	True	True

Out[140]=

LocalFeatureLength	SharedUpdateRule	LinearDotAllowed	GraphDistanceAbsent	DirectPathFeaturesAbsent
4	True	True	True	True

In[141]:=

```
policy = Q9Fit[];
Dataset[{Q9FitTest[policy]}]
Dataset[{Q9TrainingTest[policy]}]
```

Out[142]=

Weights	SearchSpace	TrainingCases	Reward	TrainingAccuracy	IntermediateLabels	Objective
{0, 0, 1, 0}	135	28	26	0.928571	0	FinalOutcomeOnly

Out[143]=

Cases	Passed	Accuracy	AllPassed
28	26	0.928571	False

In[144]:=

```
c = Q9TrainCases[];
f = MapIndexed[Function[{x, i}, Module[{r, p},
  r = Q9Decision[x["State"], policy["Weights"], x["Cue"], policy["Budget"]];
  p = Q9CasePassQ[x, policy["Weights"], policy["Budget"]]; <| "Index" → First[i],
    "Type" → x["Type"], "ExpectedStatus" → x["ExpectedStatus"], "ExpectedValue" →
    x["ExpectedValue"], "ActualStatus" → r["Status"], "ActualValue" → r["Value"],
    "WritesOK" → x["State"]["WritesOK"], "Passed" → p |>]], c];
Dataset@Select[f, !TrueQ[#[["Passed"]] &]
```

Out[146]=

Index	Type	ExpectedStatus	ExpectedValue	ActualStatus	ActualValue
8	Broken	Unknown	Unknown	Ambiguous	Ambiguous
23	Broken	Unknown	Unknown	Ambiguous	Ambiguous

In[147]:=

```
Dataset[{
  <| "Weights" → {0, 0, 1, 0}, "Score" → Q9Score[{0, 0, 1, 0}, c, 8] |>,
  <| "Weights" → {-1, 2, 2, 0}, "Score" → Q9Score[{-1, 2, 2, 0}, c, 8] |>
}]
```

Out[147]=

Weights	Score
{0, 0, 1, 0}	26
{-1, 2, 2, 0}	26

In[148]:=

```
Dataset[{{<|"Total" → policy["TotalByType"], "Passed" → policy["PassedByType"]|>}}
```

Out[148]=

	Positive	Broken	WrongCue	Conflict
Total	8	8	8	4
Passed	8	6	8	4

In[149]:=

```
ClearAll[Q9Program, Q9Replay, Q9WritePrograms,
  Q9Relabel, Q9Propagate, Q9Decision, Q9Retrieve];
```

```
Q9Program[s_Association, n_Integer, d_Integer?Positive, j_Integer, q_Integer,
  br_Integer: 3, seed_Integer: 1, tag_Integer: 0] := Module[{main, good, a,
  pt = {}, mt = {}, cp = {}, cv = {}, i, b, cur, nxt, c, tg, l, id}, SeedRandom[seed];
  main = Join[{s["K"]}, Table[s["K"] - 5 000 000 - tag * 1 000 000 - i * 10 000, {i, d - 1}],
    {s["Values"][[j]]}];
  good = RandomInteger[{1, br}, d];
  a = DeleteCases[Range[s["NCue"]], q];
  Do[cur = main[[i]];
    nxt = main[[i + 1]];
    Do[c = s["K"] - 100 000 000 - tag * 10 000 000 - i * 100 000 - b * 1000;
      tg = If[b === good[[i]], nxt, s["K"] - 200 000 000 - tag * 10 000 000 - i * 100 000 - b * 1000];
      l = If[b === good[[i]], q, a[[1 + Mod[b - 1, Length[a]]]];
      id = tag * 1 000 000 + i * 100 + b;
      AppendTo[cv, c];
      AppendTo[pt, {cur, c, tg}];
      mt = Join[mt, Q9Marks[s, c, l, n, id]];
      If[b === good[[i]], AppendTo[cp, {cur, c, tg}], {b, br}], {i, d}]; <|"MarkerLength" → n,
  "PathTriples" → pt, "MarkTriples" → mt, "CorrectPathTriples" → cp, "Candidates" →
    DeleteDuplicates[cv], "Depth" → d, "Value" → j, "Cue" → q, "Branches" → br |>];
```

```
Q9Replay[s_Association, p_Association] :=
  Module[{t}, t = Q9WriteTriples[Q9WriteTriples[s, p["PathTriples"]], p["MarkTriples"]];
  Join[t, <|"Candidates" →
    Union[Lookup[s, "Candidates", {}], Lookup[p, "Candidates", {}]] |>];
```

```
Q9WritePrograms[s_Association, p_List] := Fold[Q9Replay, s, p];
```

```

Q9Relabel[s_Association, seed_Integer] :=
Module[{v, nv, r, gm, gw}, v = Sort@Union[VertexList[s["Memory"]], VertexList[s["Work"]]];
SeedRandom[seed];
nv = Sort@RandomSample[Range[1000, 999 + 10 * Length[v]], Length[v]];
r = Thread[v → nv];
gm = Graph[VertexList[s["Memory"]] /. r, EdgeList[s["Memory"]] /. r];
gw = Graph[VertexList[s["Work"]] /. r, EdgeList[s["Work"]] /. r];
Join[s, <| "Memory" → gm, "Work" → gw, "Values" → s["Values"] /. r,
  "Actions" → s["Actions"] /. r, "Cues" → s["Cues"] /. r, "K" → s["K"] /. r,
  "AM" → s["AM"] /. r, "AT" → s["AT"] /. r, "Base" → s["Base"] /. r,
  "Candidates" → Lookup[s, "Candidates", {}] /. r |>]];

```

```

Q9Propagate[g_Graph, c_, w_List, max_Integer: 8, stop_List: {}] :=
Module[{i = Q9Index[g], v, h, raw, h2, a = <| |>, new, same, r = 0, k}, v = i["V"];
h = AssociationThread[v, ConstantArray[0, Length[v]]];
AssociateTo[h, c → 1];
AssociateTo[a, c → 0];
Do[raw = Q9Round[i, h, w];
new = Select[v, Lookup[raw, #, 0] === 1 && ! KeyExistsQ[a, #] &];
Scan[(AssociateTo[a, # → k]) &, new];
h2 = AssociationThread[v, If[MemberQ[stop, #], 0, Lookup[raw, #, 0]] & /@ v];
same = SameQ[h2, h];
h = h2;
r = k;
If[same, Break[], {k, 1, max}]; <|
"Arrival" → a, "Rounds" → r, "NodeUpdates" → r * Length[v] |>];

```

```

Q9Decision[s_Association, w_List, q_Integer, bud_Integer: 8] := Catch[
Module[{g = s["Memory"], p, a, c = s["K"], vis = {}, path = {s["K"]}, nx, t, mn, z, n, k, v},
p = Q9Propagate[g, s["Cues"][[q]], w, bud, Lookup[s, "Candidates", {}]];
a = p["Arrival"];
Do[If[MemberQ[s["Values"], c], v = First@FirstPosition[s["Values"], c];
Throw[<| "Status" → "Found", "Value" → v, "Path" → path, "Rounds" → p["Rounds"] |>]];
If[MemberQ[vis, c], Throw[<| "Status" → "Loop",
"Value" → "Loop", "Path" → path, "Rounds" → p["Rounds"] |>]];
AppendTo[vis, c];

```

```

nx = Q9Next[g, c];
If[nx === {}, Throw[<| "Status" → "Unknown",
  "Value" → "Unknown", "Path" → path, "Rounds" → p["Rounds"] |>]];
If[Length[nx] === 1, n = First[nx], t = Lookup[a, nx, Infinity];
  mn = Min[t];
  If[mn === Infinity, Throw[<| "Status" → "Unknown",
    "Value" → "Unknown", "Path" → path, "Rounds" → p["Rounds"] |>]];
  z = Pick[nx, t, mn];
  If[Length[z] > 1, Throw[<| "Status" → "Ambiguous",
    "Value" → "Ambiguous", "Path" → path, "Rounds" → p["Rounds"] |>]];
  n = First[z]];
c = n;
AppendTo[path, c], {k, 1, 3000}; <| "Status" → "Budget",
  "Value" → "Budget", "Path" → path, "Rounds" → p["Rounds"] |>]];

```

```

Q9Retrieve[s_Association, p_Association, q_Integer, max_Integer: 3000] :=
Catch[Module[{t, mem, pr, a, c, vis = {}, path, rd = 0, ev = 0, mb = 0,
  mg = 0, nx, z, mn, g, n, h, k, v}, t = Q9SW[s, Graph[s["Base"], {}]];
  mem = EdgeList[t["Memory"]];
  pr = Q9Propagate[t["Memory"], t["Cues"]][[q]],
  p["Weights"], p["Budget"], Lookup[t, "Candidates", {}]];
a = pr["Arrival"];
c = t["K"];
path = {c};
Do[If[MemberQ[t["Values"], c], v = First@FirstPosition[t["Values"], c];
  Throw[Q9Pack[t, "Found", v, path, rd, ev, mb, mg, pr, mem]]];
If[MemberQ[vis, c], Throw[Q9Pack[t, "Loop", "Loop", path, rd, ev, mb, mg, pr, mem]]];
AppendTo[vis, c];
nx = Q9Next[t["Memory"], c];
rd++;
mb = Max[mb, Length[nx]];
If[nx === {}, Throw[Q9Pack[t, "Unknown", "Unknown", path, rd, ev, mb, mg, pr, mem]]];
If[Length[nx] === 1, n = First[nx], z = Lookup[a, nx, Infinity];
  mn = Min[z];
  If[mn === Infinity,
    Throw[Q9Pack[t, "Unknown", "Unknown", path, rd, ev, mb, mg, pr, mem]]];
  g = Pick[nx, z, mn];
  mg = Max[mg, Length[g]];
  If[Length[g] > 1,
    Throw[Q9Pack[t, "Ambiguous", "Ambiguous", path, rd, ev, mb, mg, pr, mem]]];
  n = First[g]];

```

```

If[MemberQ[vis, n],
  Throw[Q9Pack[t, "Loop", "Loop", Append[path, n], rd, ev, mb, mg, pr, mem]]];
h = Q9Probe[t, c, n];
If[! TrueQ[h["OK"]],
  Throw[Q9Pack[t, "ProbeError", "ProbeError", path, rd, ev, mb, mg, pr, mem]]];
t = h["State"];
ev += h["Events"];
c = n;
AppendTo[path, c], {k, 1, max}];
Q9Pack[t, "Budget", "Budget", path, rd, ev, mb, mg, pr, mem]]];

```

In[157]:=

```

Clear[policy, c, f];
policy = Q9Fit[];
Dataset[{Q9FitTest[policy]}]
Dataset[{Q9TrainingTest[policy]}]

```

Out[159]=

Weights	SearchSpace	TrainingCases	Reward	TrainingAccuracy	IntermediateLabels	Objective
{0, 0, 1, 0}	135	28	28	1.0	0	FinalOutcomeOnly

Out[160]=

Cases	Passed	Accuracy	AllPassed
28	28	1.0	True

In[161]:=

```

c = Q9TrainCases[];
f = MapIndexed[Function[{x, i}, Module[{r, p},
  r = Q9Decision[x["State"], policy["Weights"], x["Cue"], policy["Budget"]];
  p = Q9CasePassQ[x, policy["Weights"], policy["Budget"]]; <|"Index" → First[i],
    "Type" → x["Type"], "ExpectedStatus" → x["ExpectedStatus"], "ActualStatus" →
    r["Status"], "WritesOK" → x["State"]["WritesOK"], "Passed" → p |>]], c];
Dataset@Select[f, ! TrueQ[##["Passed"]] &]

```

Out[163]=



In[164]:=

```

Q9DepthTest[policy, 4, Range[1, 4], 4, 3]
Q9DepthTest[policy, 6, Range[5, 12], 4, 3]
Q9DepthTest[policy, 8, Range[5, 12], 4, 3, True, True]

```


Out[164]=

MarkerLength	Depth	Value	Cue	Branches	Permuted	Relabeled	ExpectedFrozenEdges	FrozenEdg
4	1	1	1	3	False	False	18	18
4	1	2	2	3	False	False	18	18
4	1	3	3	3	False	False	18	18
4	1	4	4	3	False	False	18	18
4	2	1	2	3	False	False	36	36
4	2	2	3	3	False	False	36	36
4	2	3	4	3	False	False	36	36
4	2	4	5	3	False	False	36	36
4	3	1	3	3	False	False	54	54
4	3	2	4	3	False	False	54	54
4	3	3	5	3	False	False	54	54
4	3	4	6	3	False	False	54	54
4	4	1	4	3	False	False	72	72
4	4	2	5	3	False	False	72	72
4	4	3	6	3	False	False	72	72
4	4	4	7	3	False	False	72	72

◀ < columns 1–10 of 18 > ▶

Out[165]=

MarkerLength	Depth	Value	Cue	Branches	Permuted	Relabeled	ExpectedFrozenEdges	FrozenF
6	5	1	5	3	False	False	120	120
6	5	2	6	3	False	False	120	120
6	5	3	7	3	False	False	120	120
6	5	4	8	3	False	False	120	120
6	6	1	6	3	False	False	144	144
6	6	2	7	3	False	False	144	144
6	6	3	8	3	False	False	144	144
6	6	4	1	3	False	False	144	144
6	7	1	7	3	False	False	168	168
6	7	2	8	3	False	False	168	168
6	7	3	1	3	False	False	168	168
6	7	4	2	3	False	False	168	168
6	8	1	8	3	False	False	192	192
6	8	2	1	3	False	False	192	192
6	8	3	2	3	False	False	192	192
6	8	4	3	3	False	False	192	192
6	9	1	1	3	False	False	216	216
6	9	2	2	3	False	False	216	216
6	9	3	3	3	False	False	216	216
6	9	4	4	3	False	False	216	216

rows 1-20 of 32

columns 1-10 of 18

Out[166]=

MarkerLength	Depth	Value	Cue	Branches	Permuted	Relabeled	ExpectedFrozenEdges	FrozenEdges
8	5	1	5	3	True	True	150	150
8	5	2	6	3	True	True	150	150
8	5	3	7	3	True	True	150	150
8	5	4	8	3	True	True	150	150
8	6	1	6	3	True	True	180	180
8	6	2	7	3	True	True	180	180
8	6	3	8	3	True	True	180	180
8	6	4	1	3	True	True	180	180
8	7	1	7	3	True	True	210	210
8	7	2	8	3	True	True	210	210
8	7	3	1	3	True	True	210	210
8	7	4	2	3	True	True	210	210
8	8	1	8	3	True	True	240	240
8	8	2	1	3	True	True	240	240
8	8	3	2	3	True	True	240	240
8	8	4	3	3	True	True	240	240
8	9	1	1	3	True	True	270	270
8	9	2	2	3	True	True	270	270
8	9	3	3	3	True	True	270	270
8	9	4	4	3	True	True	270	270

rows 1-20 of 32 columns 1-10 of 18

In[167]:=

```

a4 = Normal@Q9DepthTest[policy, 4, Range[1, 4], 4, 3];
a6 = Normal@Q9DepthTest[policy, 6, Range[5, 12], 4, 3];
a8 = Normal@Q9DepthTest[policy, 8, Range[5, 12], 4, 3, True, True];

Dataset[{
  <|"Test" → "Length4", "Cases" → Length[a4], "Passed" → Count[Lookup[a4, "Passed"], True],
    "AllPassed" → And @@ Lookup[a4, "Passed"]|>,
  <|"Test" → "Length6", "Cases" → Length[a6], "Passed" → Count[Lookup[a6, "Passed"], True],
    "AllPassed" → And @@ Lookup[a6, "Passed"]|>,
  <|"Test" → "Length8PermRelabel", "Cases" → Length[a8], "Passed" →
    Count[Lookup[a8, "Passed"], True], "AllPassed" → And @@ Lookup[a8, "Passed"]|>
}]

```

Out[170]=

Test	Cases	Passed	AllPassed
Length4	16	16	True
Length6	32	32	True
Length8PermRelabel	32	32	True

In[171]:=

```
Dataset@Select[Join[a4, a6, a8], !TrueQ[#[["Passed"]] &]
```

Out[171]=



In[172]:=

```
Dataset[{Q9HistoryTest[policy]]
Dataset[{Q9BrokenTest[policy]]
Dataset[{Q9ConflictTest[policy]]
Dataset[{Q9WrongCueTest[policy]]
Dataset[{Q9NoiseTest[policy]]
Dataset[{Q9CompressionTest[policy]]
```

Out[172]=

SameActive	SameVertices	DifferentFrozen	RecallA	RecallB	ExecutedA	ExecutedB	ActiveOnlyA	Act
True	True	True	1	3	1	3	Unknown	Unl

K < columns 1–10 of 12 > X

Out[173]=

Status	Recall	Selected	Executed	AllPassed
Unknown	Unknown	NONE	Unknown	True

Out[174]=

Status	Recall	Selected	MaxGood	AllPassed
Ambiguous	Ambiguous	NONE	2	True

Out[175]=

StoredCue	QueryCue	Status	Selected	AllPassed
2	3	Unknown	NONE	True

Out[176]=

Noise	Status	Recall	Executed	AllPassed
10	Found	2	2	True

Out[177]=

Generated	ExpectedUnique	Unique	CompressionRate	Recall	Executed	AllPassed
7680	240	240	0.96875	3	3	True

In[178]:=

```
tests = <|
  "History" → Q9HistoryTest[policy],
  "Broken" → Q9BrokenTest[policy],
  "Conflict" → Q9ConflictTest[policy],
  "WrongCue" → Q9WrongCueTest[policy],
  "Noise" → Q9NoiseTest[policy],
  "Compression" → Q9CompressionTest[policy]
|>;
```

```
Dataset@KeyValueMap[
  <|"Test" → #1, "AllPassed" → Lookup[#2, "AllPassed", False]|> &,
  tests
]
```

```
And @@ Lookup[Values[tests], "AllPassed"]
```

Out[179]=

Test	AllPassed
History	True
Broken	True
Conflict	True
WrongCue	True
Noise	True
Compression	True

Out[180]=

True

In[181]:=

```
z0 = Normal@Q9Trials[policy, 30, 12, 4, 20 260 805];
Counts[Lookup[z0, "Passed"]]
```

Out[182]=

```
<| True → 30 |>
```

In[183]:=

```
z1 = Normal@Q9Trials[policy, 30, 12, 4, 20 260 805, True, True];
Counts[Lookup[z1, "Passed"]]
```

Out[184]=

```
<| True → 30 |>
```

```

In[185]:=
e1 = Q9Batch[policy, 1]; Counts[Lookup[e1, "Passed"]]

Out[185]=
<| True → 5 |>

In[186]:=
e2 = Q9Batch[policy, 2]; Counts[Lookup[e2, "Passed"]]

Out[186]=
<| True → 5 |>

In[187]:=
e3 = Q9Batch[policy, 3]; Counts[Lookup[e3, "Passed"]]

Out[187]=
<| True → 5 |>

In[188]:=
e4 = Q9Batch[policy, 4]; Counts[Lookup[e4, "Passed"]]

Out[188]=
<| True → 5 |>

In[189]:=
e5 = Q9Batch[policy, 5]; Counts[Lookup[e5, "Passed"]]

Out[189]=
<| True → 5 |>

In[190]:=
e6 = Q9Batch[policy, 6]; Counts[Lookup[e6, "Passed"]]

Out[190]=
<| True → 5 |>

In[191]:=
e7 = Q9Batch[policy, 7]; Counts[Lookup[e7, "Passed"]]

Out[191]=
<| True → 5 |>

In[192]:=
e8 = Q9Batch[policy, 8]; Counts[Lookup[e8, "Passed"]]

Out[192]=
<| True → 5 |>

In[193]:=
e9 = Q9Batch[policy, 9]; Counts[Lookup[e9, "Passed"]]

Out[193]=
<| True → 5 |>

In[194]:=
e10 = Q9Batch[policy, 10]; Counts[Lookup[e10, "Passed"]]

Out[194]=
<| True → 5 |>

```

In[195]:=

```
z2 = Join[e1, e2, e3, e4, e5, e6, e7, e8, e9, e10];  
Counts[Lookup[z2, "Passed"]]
```

Out[196]=

```
<| True → 50 |>
```